

SKILL WEEK-2

Step 1 : Create table

```
3 • CREATE TABLE orders (  
4     ord_no INT PRIMARY KEY,  
5     purch_amt DECIMAL(10,2),  
6     ord_date DATE,  
7     customer_id INT,  
8     salesman_id INT  
9 );  
10
```

Output

Action Output

#	Time	Action	Message
✓ 1	12:54:48	USE 2520080069_Ananya	0 row(s) affected
✓ 2	12:55:04	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_a...	0 row(s) affected

Step 2 : Insert data

```
12 • INSERT INTO orders VALUES  
13 (70001,150.50,'2012-10-05',3005,5002),  
14 (70009,270.65,'2012-09-10',3001,5005),  
15 (70002,65.26,'2012-10-05',3002,5001),  
16 (70004,110.50,'2012-08-17',3009,5003),  
17 (70007,948.50,'2012-09-10',3005,5002),  
18 (70005,2400.60,'2012-07-27',3007,5001),  
19 (70008,5760.00,'2012-09-10',3002,5001),  
20 (70010,1983.43,'2012-10-10',3004,5006),  
21 (70003,2480.40,'2012-10-10',3009,5003),  
22 (70012,250.45,'2012-06-27',3008,5002),  
23 (70011,75.29,'2012-08-17',3003,5007),  
24 (70013,3045.60,'2012-04-25',3002,5001);  
25  
26
```

toggle aut

Context Help Snipp

Output

Action Output

#	Time	Action	Message
✓ 1	12:54:48	USE 2520080069_Ananya	0 row(s) affected
✓ 2	12:55:04	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_a...	0 row(s) affected
✓ 3	12:56:22	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,50...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0

Query 1 : WHERE CLAUSE

Finance wants all orders whose purchase amount is greater than \$2,000.

```
27 • SELECT * FROM orders
28     WHERE purch_amt > 2000;
29
```

Result Grid					
		Filter Rows:	Edit:		Export/Import:
ord_no	purch_amt	ord_date	customer_id	salesman_id	
70003	2480.40	2012-10-10	3009	5003	
70005	2400.60	2012-07-27	3007	5001	
70008	5760.00	2012-09-10	3002	5001	
70013	3045.60	2012-04-25	3002	5001	
*	NULL	NULL	NULL	NULL	

Query 2 : WHERE CLAUSE

Management wants all orders placed on 2012-09-10.

```
30 • SELECT * FROM orders
31     WHERE ord_date='2012-09-10';
```

Result Grid					
		Filter Rows:	Edit:		Export/Import:
ord_no	purch_amt	ord_date	customer_id	salesman_id	
70007	948.50	2012-09-10	3005	5002	
70008	5760.00	2012-09-10	3002	5001	
70009	270.65	2012-09-10	3001	5005	
*	NULL	NULL	NULL	NULL	

Query 3 : WHERE CLAUSE

Find all orders handled by salesman 5001

```
33 • SELECT * FROM orders
34 WHERE salesman_id=5001;
35
```

Result Grid					
Filter Rows:					
Edit: Export/Import:					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70002	65.26	2012-10-05	3002	5001
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
*	NULL	NULL	NULL	NULL	NULL

Query 4 : ORDER BY

CEO wants to see orders ranked from highest amount to lowest amount.

```
36 • SELECT *
37 FROM orders
38 ORDER BY purch_amt DESC;
39
```

Result Grid					
Filter Rows:					
Edit: Export/Import:					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70010	1983.43	2012-10-10	3004	5006
	70007	948.50	2012-09-10	3005	5002
	70009	270.65	2012-09-10	3001	5005
	70012	250.45	2012-06-27	3008	5002
	70001	150.50	2012-10-05	3005	5002
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007
	70002	65.26	2012-10-05	3002	5001
*	NULL	NULL	NULL	NULL	NULL

orders 4 x

Query 5 : ORDER BY

Operations team wants oldest orders first.

```
40 • SELECT *
41 FROM orders
42 ORDER BY ord_date;
```

Result Grid					
		Filter Rows:	Edit:		
ord_no	purch_amt	ord_date	customer_id	salesman_id	
70013	3045.60	2012-04-25	3002	5001	
70012	250.45	2012-06-27	3008	5002	
70005	2400.60	2012-07-27	3007	5001	
70004	110.50	2012-08-17	3009	5003	
70011	75.29	2012-08-17	3003	5007	
70007	948.50	2012-09-10	3005	5002	
70008	5760.00	2012-09-10	3002	5001	
70009	270.65	2012-09-10	3001	5005	
70001	150.50	2012-10-05	3005	5002	
70002	65.26	2012-10-05	3002	5001	
70003	2480.40	2012-10-10	3009	5003	
70010	1983.43	2012-10-10	3004	5006	
NULL	NULL	NULL	NULL	NULL	

Query 6 : SUM()

Finance department wants total revenue generated.

```
44 • SELECT SUM(purch_amt) AS total_revenue
45 FROM orders;
```

Result Grid	
Filter Rows:	
Export:	
Wr	
total_revenue	
17541.18	

Query 7 : AVG()

Management wants average order value

```
48 • SELECT AVG(purch_amt) AS average_order
49 FROM orders;
```

Result Grid		Filter Rows:	Export:	Wrap C
	average_order			
▶	1461.765000			

Query 8 : MAX()

Find the highest order amount

```
51 • SELECT MAX(purch_amt) AS highest_order
52 FROM orders;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	highest_order			
▶	5760.00			

Query 9 : MIN()

Find the lowest order amount

```
54 • SELECT MIN(purch_amt) AS lowest_order
55 FROM orders;
```

Result Grid		Filter Rows:	Export:
	lowest_order		
▶	65.26		

Query 10 : COUNT()

Find total number of orders

```
57 • SELECT COUNT(*) AS total_orders
58 FROM orders;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_orders			
▶	12			

Query 11 : GROUP BY

Sales director wants total sales generated by each salesman

```
60 • SELECT salesman_id,  
61     SUM(purch_amt) AS total_sales  
62     FROM orders  
63     GROUP BY salesman_id;  
64
```

Result Grid			Filter Rows:	Export:	Wrap Cell C
	salesman_id	total_sales			
▶	5002	1349.45			
	5001	11271.46			
	5003	2590.90			
	5005	270.65			
	5006	1983.43			
	5007	75.29			

Query 12 : GROUP BY

Marketing wants total purchases made by each customer.

```
65 • SELECT customer_id,  
66     SUM(purch_amt) AS total_purchase  
67     FROM orders  
68     GROUP BY customer_id;
```

Result Grid			Filter Rows:	Export:	Wr
	customer_id	total_purchase			
▶	3005	1099.00			
	3002	8870.86			
	3009	2590.90			
	3007	2400.60			
	3001	270.65			
	3004	1983.43			
	3003	75.29			
	3008	250.45			

Query 13 : GROUP BY + MAX()

Finding the highest order placed by each customer

69

```
70 • SELECT customer_id,  
71     MAX(purch_amt) AS highest_purchase  
72     FROM orders  
73     GROUP BY customer_id;  
74
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_id	highest_purchase			
▶	3005	948.50	948.50		
	3002	5760.00			
	3009	2480.40			
	3007	2400.60			
	3001	270.65			
	3004	1983.43			
	3003	75.29			
	3008	250.45			

Query 14 : GROUP BY + HAVING

Company rewards salesmen whose total sales exceeds \$3,000

74

```
75 • SELECT salesman_id,  
76     SUM(purch_amt) AS total_sales  
77     FROM orders  
78     GROUP BY salesman_id  
79     HAVING SUM(purch_amt) > 3000;  
80
```

Result Grid			Filter Rows:	Export:
	salesman_id	total_sales		
	5001	11271.46		

Query 15 : GROUP BY + HAVING

Find customers whose total purchases exceed \$2,500.

```
81 • SELECT customer_id,  
82     SUM(purch_amt) AS total_purchase  
83     FROM orders  
84     GROUP BY customer_id  
85     HAVING SUM(purch_amt) > 2500;  
86
```

Result Grid		Filter Rows:	Export:	Wrap Cell Co
	customer_id	total_purchase		
▶	3002	8870.86		
	3009	2590.90		

Query 16 : GROUP BY + HAVING

Find customers who placed more than one order

```
88 • SELECT customer_id,  
89     COUNT(*) AS total_orders  
90     FROM orders  
91     GROUP BY customer_id  
92     HAVING COUNT(*) > 1;  
93
```

Result Grid		Filter Rows:	Exp
	customer_id	total_orders	
▶	3005	2	
	3002	3	
	3009	2	

Query 17 : GROUP BY + HAVING + ORDER BY

Management wants top customers ranked by spending

```
94 • SELECT customer_id,  
95 SUM(purch_amt) AS total_purchase  
96 FROM orders  
97 GROUP BY customer_id  
98 HAVING SUM(purch_amt) > 1000  
99 ORDER BY total_purchase DESC;  
100
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content
customer_id	total_purchase		
3002	8870.86		
3009	2590.90		
3007	2400.60		
3004	1983.43		
3005	1099.00		

Query 18 : BETWEEN + HAVING

Find customers whose maximum purchase is between \$2,000 and \$6,000.

```
101 • SELECT customer_id,  
102 MAX(purch_amt) AS max_purchase  
103 FROM orders  
104 GROUP BY customer_id  
105 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;  
106
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content
customer_id	max_purchase		
3002	5760.00		
3009	2480.40		
3007	2400.60		

Query 19 : COUNT + HAVING

Find salesmen who handled at least two orders

```
107 • SELECT salesman_id,  
108      COUNT(*) AS total_orders  
109      FROM orders  
110      GROUP BY salesman_id  
111      HAVING COUNT(*) >= 2;  
112  
113
```

Result Grid			Filter Rows:	Exp
	salesman_id	total_orders		
▶	5002	3		
	5001	4		
	5003	2		

Query 20 : MAX + GROUP BY + HAVING

Management wants to daily highest purchases above \$2000

```
114 • SELECT ord_date,  
115           MAX(purch_amt) AS highest_purchase  
116           FROM orders  
117           GROUP BY ord_date  
118           HAVING MAX(purch_amt) > 2000;  
119
```

Result Grid			Filter Rows:	Export:	Wrap Cell Cont
	ord_date	highest_purchase			
▶	2012-10-10	2480.40			
	2012-07-27	2400.60			
	2012-09-10	5760.00			
	2012-04-25	3045.60			

